

COL7(6)83 Assignment 3

Due date: 28 October

This assignment deals with image transforms, wavelets, and image compression. All tasks have to be implemented yourself by working with the array of image pixels, unless otherwise specified in the problem. You are permitted to perform all tasks using intensity images.

Some standard test images used in image processing are provided. You may use these or other images of your own choice for your results.

As with Assignment 2, if you are in a group of two, you are required to complete one of the three optional tasks below; if you are working individually, no optional task is required. In either case, doing one more optional task (beyond the 1 or 0 required) will give some extra marks; doing two more will not increase the marks further.

1. As discussed in class, Gaussian pyramids are useful for stitching multiple images together. Obtain two such equal-sized images f and g that you would like to stitch, e.g. blending between an apple and an orange, or swapping faces of two people, or mixing day and night views of an outdoor scene. (You can also try constructing a panorama from multiple overlapping images of a very wide/tall scene, but then you will have to perform some geometric transformations to align the images first.)
 - a. Construct a Gaussian pyramid of f , up to some chosen number of levels. A common approach is to convolve the image with a filter $w = \frac{1}{16}[1, 4, 6, 4, 1]$ and then sample every alternate pixel. Show the resulting stack of images in the Gaussian pyramid.
 - b. Then, construct the Laplacian pyramid by subtracting from each level the prediction obtained from the next upper level. (Usually, prediction is done by inserting zeros, multiplying by 2, and convolving with w again: $[f[0], f[1], f[2], \dots] \mapsto w * [2f[0], 0, 2f[1], 0, 2f[2], 0, \dots]$.) Show the images in the Laplacian pyramid as well. Verify that you are able to reconstruct f exactly using only the Laplacian pyramid.
 - c. Manually define a binary mask m of the same size as f and g , indicating which regions of the output should come from which image. Show the image $(1 - m)f + mg$, which stitches the two images with a hard transition.

- d. Create a Gaussian pyramid of m , and use it to blend the levels of the Laplacian pyramids of f and g . Use the blended Laplacian pyramid to reconstruct the final image, which should show a smooth transition.
2. Take an $N \times N$ image whose size is a power of two, $N = 2^k$. This will make it easier to apply the Haar transform.
- Prove mathematically that if f is any 1D array of length $N = 2^k$ and t is its Haar transform, then $\|t\| = \|f\|$. (This fact is sufficient to imply that the Haar transform is orthogonal.) Then, implement functions to perform the 1D Haar transform and its inverse, and verify numerically that this result holds on some interesting test signal (e.g. the middle row of your chosen $N \times N$ image).
 - Implement functions that perform the 2D forward/inverse Haar transforms of an image by applying your 1D functions on all the rows and columns. Show the input image f and the Haar transform t . (Since t will contain both positive and negative values, it will be better to show $t + I_{\max}/2$.)
 - The Haar transform, like other wavelet transforms, has the useful property that much of the image information is concentrated into a small number of coefficients in t . Explore this phenomenon as follows: Find the top $N^2/16$ coefficients in the Haar transform, i.e. the ones with the largest absolute value, and zero out the rest. Perform the inverse Haar transform and visualize the approximated image. For comparison, show the results of downsampling the image to $N/4 \times N/4$ (thus again using $N^2/16$ coefficients to represent the image). Repeat the experiment, retaining only the top $N^2/256$ Haar coefficients vs. downsampling to $N/16 \times N/16$.
3. Let's repeat part of Assignment 2's problem 4 on denoising, but this time with wavelet denoising. Choose a photographic image f whose width and height are powers of 2, and add Gaussian noise to obtain a degraded image g with PSNR = 20 dB.
- Compute the Haar wavelet transforms of both f and g , up to some chosen number of levels (at least 4). Note that this is slightly different from the separable Haar transform from problem 2, since you only recurse on the upper-left quadrant at each level. Visualize the resulting coefficient arrays as images, adding 128 to the detail coefficients (everything except the coarsest-scale approximation coefficients) since they will be both

positive and negative. Also, plot the histogram of only the detail coefficients for f , g , and the noise $\eta = g - f$.

- b. Perform hard thresholding with a threshold t , and record the PSNR of the denoised image as a function of t . Do the same with soft thresholding, and make a single plot showing the PSNR of both methods with t on the x -axis. Show the denoised images with the best PSNR for both hard and soft thresholding.
 - c. **Optional:** Implement the Daubechies db2 wavelet transform, using the scaling and wavelet function coefficients given in the slides and the textbook. You will need to choose some padding scheme for both the forward and inverse transforms, and you may observe some artifacts near the boundaries; this is acceptable as long as the inverse transform exactly recovers the intensities in the interior of the image. Repeat tasks (a) and (b) using db2 wavelets, and comment on the quality relative to Haar wavelets.
4. In this problem, you will perform block DCT coding, like in JPEG. For testing, choose two different images: (i) an image f_1 containing large regions of the same intensity, such as a diagram, illustration, or cartoon, and (ii) a photographic image f_2 . Crop the images so their height and width is a multiple of 16.
- a. Implement a 16×16 block DCT, to transform the image into a collection of coefficient arrays. Also implement the inverse transformation, and verify you get back the same image. You may use a built-in DCT implementation.
 - b. Consider a very simple lossy compression scheme, in which certain coefficients are discarded and replaced with zeroes. Suppose we would like to discard as many coefficients as possible while ensuring that the resulting PSNR of the block is no less than 20 dB, i.e. $\text{RMSE} \leq 0.1I_{\max}$. Explain how to do so without performing multiple IDCTs, using the fact that the DCT is orthogonal. Implement your strategy, and for each of f_1 and f_2 , show the resulting decoded image $\hat{f}_i$, report the PSNR of the entire image and the total number of nonzero coefficients, and show the error image $|f_i - \hat{f}_i|$.
 - c. This approach could be applied to the DCT of the entire image, instead of per-block DCTs, but would it perform better or worse? Repeat (b) using the full DCTs of the images f_1 and f_2 , and show the same results. Discuss the pros and cons in your report.

- d. **Optional:** As discussed in class, recent image compression formats allow more options of predictive coding on blocks, for example replicating the row immediately above the block, and replicating the column immediately to the left. The prediction error is then encoded using lossy DCT. Implement this encoding strategy (choosing the predictor with the lowest RMSE for each block), as well as the corresponding decoder. Show the same results as in (b): the decoded image, PSNR, nonzero coefficients, and error image. Also show the outcome if the prediction error is computed using the rows/columns of the original image, rather than the decoded blocks.
5. The PNG file format uses a combination of predictive coding, an LZW-like algorithm, and Huffman coding. Use the same two images f_1 and f_2 from problem 4 for testing. (Note: PNG actually uses LZ77, an older algorithm by Ziv & Lempel, because LZW was patent-encumbered until 2004. However, we will use LZW since that's what we have studied in class.)
- Assume that the intensity of the top-left pixel is coded as-is, the remaining pixels in the topmost row and leftmost column are predicted using the pixel to the left and above respectively, and all other pixels are predicted using the Paeth predictor. Compute and display the error images e_1 (for f_1) and e_2 (for f_2) showing the prediction error at each pixel. Also show the error histogram, and report the standard deviation and the entropy (with the convention that $p \log p = 0$ if $p = 0$).
 - Implement a decoder which, when given one of the error images from (a), can recover the original image exactly. Verify it on e_1 and e_2 .
 - Implement an LZW encoder and decoder for lists of ~~8-bit~~ integers. To apply it in conjunction with predictive coding, you can either initialize the codebook with all possible errors (-255 to 255), or encode the errors modulo 255 (e.g. if $f = 0$ and $\dot{f} = 1$ then $e = 0 - 1 = 255$, and while decoding $\dot{f} + e = 1 + 255 = 0$). First test your LZW implementation by encoding and decoding a list of N zeros — you will run into the one corner case in the LZW decoding algorithm! Also, for this case of a constant sequence, find the length of the LZW output as big-O of the input length N , and prove it mathematically.
 - After the error values are LZW-compressed, we will further compress them using Huffman coding. Implement a Huffman encoder that takes a set of symbols and their corresponding probabilities, and builds a code table that associates each symbol with a list of bits. Also implement a Huffman decoder that takes the code table and a list of bits, and decodes it into a list of symbols. Build a Huffman code for the codewords output by LZW, and use it to encode the entire image as a list of bits. Also, for either f_1 or f_2 ,

compute the entropy of the codeword distribution (after prediction + LZW), and compare it with the average Huffman code length.

- e. Implement an encoder that puts all the stages together, taking an image f of size $M \times N$ as input, and producing a tuple containing the Huffman code table T and the list of encoded bits $\mathbf{b}$. Also implement a decoder that takes $M, N, T, \mathbf{b}$ as input and reconstructs the original image f . For each of f_1 and f_2 , report the compression ratio expected due to prediction ($8/H$, where H is the entropy of e), the approximate compression ratio due to LZW (input length MN / output length), and the actual compression ratio of the entire process ($8MN$ / number of output bits).
- f. **Optional:** The actual PNG format allows the encoder to choose, for each row, which of the 4 prediction schemes discussed in lecture 20 to use, or to use no prediction at all. Modify your encoder to try all 5 possibilities when encoding each row, and choose the best one. It will also have to output, at the start of the row, which prediction scheme is used, and the decoder will have to take that into account. After implementing this change, redo part (e) and report the improvement in compression. The PNG format also allows the use of run length encoding instead of a Lempel-Ziv scheme. Implement this as another option in the encoder, outputting two symbols for each run (run length and intensity) which will then go through Huffman coding. As before, report whether this improves the compression or not.

SUBMISSION

You will submit a zip file that contains all your code for the assignment, and a PDF report that includes your results for each of the assignment problems. In the text of the report, also provide all relevant information for each part.

All images should ideally be saved in PNG format, which is lossless and so does not cause any information loss (other than quantization if the intensities are not already in 8-bit format). JPEG is permitted if your submission PDF is becoming too big for the upload limit.

Your assignment should be submitted on [Gradescope](#) before midnight on the due date. There is a separate submission form for the report and the code. Only one person in a group needs to submit. Late days are counted with a quantization of 0.5 days: if you cannot finish the assignment by midnight, get some sleep and submit by noon the following day.

Separately, each of you must individually submit a short response in a separate form regarding how much you and your partner contributed to the work in this assignment. Your assignment marks will not be released until you submit the form.

